IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Muchos árboles votan mejor que uno

Teoría, práctica y, al final, material de consulta

Sprint 8 · S08_02 · teoría breve, 30 min de práctica y material de consulta

LA IDEA

Un árbol opina; un bosque vota

Como pedir opinión a cinco compañeros en lugar de a uno: si cada uno mira cosas distintas, **los fallos de unos los tapan los otros**.

reserva nueva	árbol 1	árbol 2	árbol 3	árbol 4	árbol 5	votos «cancela»	bosque
A · 10 días, con depósito	0	0	0	0	0	0 de 5	0,0
B · 22 días, sin depósito	1	1	1	1	0	4 de 5	0,8
C · 38 días, con depósito	0	1	0	1	1	3 de 5	0,6
D · 80 días, sin depósito	1	1	1	1	1	5 de 5	1,0

Cada árbol dice 1 (cancela) o 0 (no cancela). La probabilidad del bosque es la **proporción de votos**: 3 de 5 = 0,6.

AZAR Nº 1 · BAGGING

Cada árbol ve una muestra distinta

Se sortean reservas **con reemplazo**: se saca una, se anota y se devuelve a la bolsa. Así unas salen repetidas y otras no salen. Ejemplo con 8 reservas, A a H:

árbol	muestra de 8 reservas (con repetidas)	se queda fuera (OOB)
árbol 1	A · B · D · E · G · G · H · H	C y F
árbol 2	B · C · C · C · D · D · G · G	A, E, F y H
árbol 3	A · A · E · E · F · G · G · G	B, C, D y H

≈ 37 %

Fuera de cada muestra queda en torno al **37 %**: con 8 reservas de media el 34,4 %, con 100 el 36,6 % y, al crecer, tiende a **36,8 %**.

OOB = out-of-bag («fuera de la bolsa»)

Las reservas que un árbol no vio son su **examen sorpresa**. La G entró en los tres árboles: ninguno la tiene como examen.

AZAR Nº 2 · VARIABLES, Y LOS MANDOS DEL BOSQUE

Cada pregunta mira solo algunas columnas

Si todos los árboles pudieran elegir entre todas las columnas, casi todos preguntarían primero por la misma y saldrían **casi iguales**. Con **max_features**, cada pregunta ve solo un grupo al azar.

Mando	Qué controla	En el cuaderno
n_estimators	Número de árboles. Más árboles casi nunca perjudica: solo cuesta tiempo	300
min_samples_leaf	Mínimo de reservas en cada hoja: frena el sobreajuste de cada árbol	5
max_depth	Profundidad máxima de cada árbol	sin límite
max_features	Columnas que ve cada pregunta (por defecto, la raíz cuadrada del total)	5 de 34

Tras el one-hot hay 34 columnas: la raíz cuadrada de 34 es 5,8 y cada pregunta elige entre 5.

VALIDACIÓN GRATIS · OOB

Cada árbol tiene un examen que no ha visto

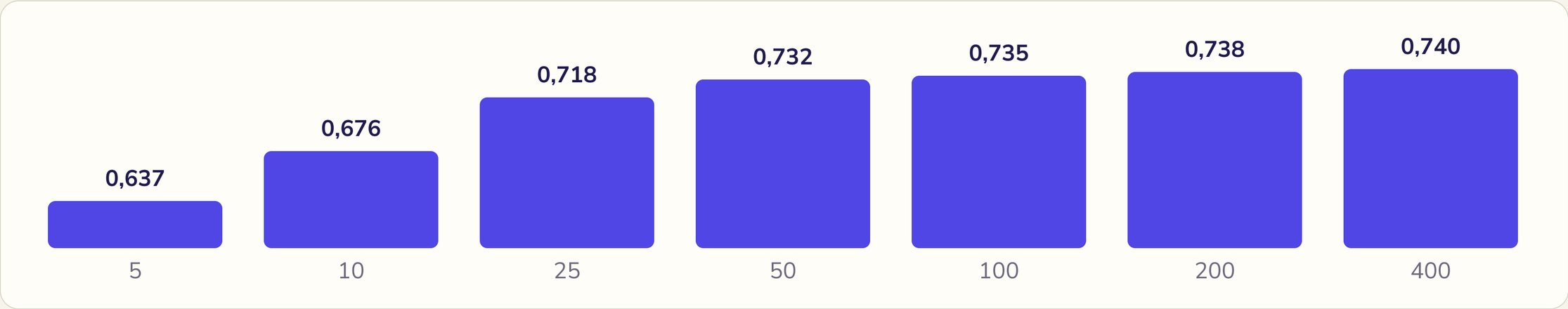
La valoración OOB de una reserva sale **solo de los árboles que no la vieron**: una nota honesta sin separar datos.

Árboles del bosque	Reservas de train sin valoración OOB	AUC out-of-bag
5	487 de 4800 (10,1 %)	0,637
10	58 de 4800 (1,2 %)	0,676
25	0	0,718

Con pocos árboles, algunas reservas nunca quedan fuera: no tienen examen y la nota baja.

CUÁNTOS ÁRBOLES

Más árboles ayudan, hasta que se estabiliza



AUC out-of-bag según el número de árboles. Las barras parten de 0,60 para apreciar la diferencia.

De 5 a 25 árboles, una subida enorme; de 100 a 400, casi nada: rendimientos decrecientes.

IMPORTANCIA DE VARIABLES

Baraja una columna y mira cuánto empeora

Ejemplo: 600 reservas inventadas	Importancia de impureza (train)	Caída del AUC al barajar (test)
antelacion_dias (sí influye)	0,815	0,345
dia_del_mes (puro ruido)	0,185	–0,012

La impureza da un 18 % de importancia a una columna de ruido; la permutación la deja en cero.

Cómo se calcula

Se **baraja** una columna del test y se mide cuánto cae el AUC.

Cómo se lee

Una caída **cerca de 0** (o negativa) significa que la variable no aporta.

COSTE Y EXPLICABILIDAD

¿Compensa un bosque?

Tiempo

300 árboles tardan **decenas de veces más** en ajustarse que uno solo, y también en predecir.

Explicabilidad

300 árboles **no se leen**. Una logística da odds ratios y se explica al cliente.

Ganancia

Si el bosque mejora solo unas centésimas de AUC, quizá **no compensa**.

Compara siempre AUC + coste + explicabilidad. Más complejo no siempre es mejor.

PRÁCTICA · CUADERNO S08_02

Lo que haremos en el cuaderno

Parte	Qué haremos	Dónde
Preparación y ejemplo	Cargar los datos y ver qué reservas quedan fuera de una muestra bootstrap	Clase
Ejercicio 1	Un bosque de 300 árboles frente a un árbol podado	Clase
Ejemplo	AUC out-of-bag según el número de árboles	Clase
Ejercicio 2	Rendimientos decrecientes: de 5 a 400 y de 100 a 400 árboles	Clase
Ejemplo	Importancia por permutación en el pipeline completo	Casa
Ejercicio 3	La variable principal y las variables irrelevantes	Casa
Ejemplo	Tiempo de ajuste: árbol frente a bosque	Casa
Mini-reto	Duelo: árbol, bosque y regresión logística	Casa

Cada ejercicio: **lee** → **escribe** → **ejecuta** → **comprueba el ✔**. La solución está plegada: ábrela tras intentarlo.

CONSULTA

Material de consulta: código y aclaraciones

Cuándo abrirlo

Después de intentar el ejercicio, o cuando algo no te salga. En clase no se proyecta.

Cómo está ordenado

En el **mismo orden que el cuaderno**: ejercicios 1 a 3, tiempo de ajuste y mini-reto. Cada uno trae su código y sus aclaraciones.

Ejercicio	Tema	Diapositivas
1	Bosque frente a árbol	11 y 12
2	Número de árboles	13 y 14
3 (casa)	Importancia por permutación	15 y 16
Ejemplo (casa)	Tiempo de ajuste	17 y 18
Mini-reto (casa)	Duelo con la logística	19 y 20

CONSULTA · EJERCICIO 1

Un bosque frente a un árbol podado (ejercicio 1)

```
bosque = RandomForestClassifier(n_estimators=300, min_samples_leaf=5, random_state=SEMILLA, n_jobs=2).fit(Xtr, y_train)
auc_test_bosque = roc_auc_score(y_test, bosque.predict_proba(Xte)[:, 1])    # 0.734
arbol = DecisionTreeClassifier(min_samples_leaf=80, random_state=SEMILLA).fit(Xtr, y_train)
auc_test_arbol = roc_auc_score(y_test, arbol.predict_proba(Xte)[:, 1])      # 0.716
```

0,716

AUC en test del árbol podado

0,734

AUC en test del bosque

El bosque gana por poco: +0,018 de AUC.

CONSULTA · ACLARACIONES · EJERCICIO 1

Leer el bosque (ejercicio 1)

n_estimators=300

El número de **árboles** del bosque. **min_samples_leaf=5** frena el sobreajuste de cada uno.

n_jobs=2

Entrena con **2 núcleos** a la vez: más rápido, mismo resultado.

Cada árbol es peor que el podado

Los del bosque tienen hojas de 5 reservas (frente a 80), así que **cada uno sobreajusta más**.

Por qué el bosque gana

Sus errores **no coinciden**: al votar 300 árboles distintos el ruido se cancela y baja la varianza.

CONSULTA · EJERCICIO 2

Rendimientos decrecientes (ejercicio 2)

```
n_arboles = [5, 10, 25, 50, 100, 200, 400]
for k in n_arboles:
    b = RandomForestClassifier(n_estimators=k, min_samples_leaf=5, oob_score=True, random_state=SEMILLA, n_jobs=2).fit(Xtr, y_train)
    oob_auc.append(roc_auc_score(y_train, np.nan_to_num(b.oob_decision_function_[:, 1], nan=0.5)))
mejora_5_a_400 = oob_auc[-1] - oob_auc[0]                          # 0.103
mejora_100_a_400 = oob_auc[-1] - oob_auc[n_arboles.index(100)]     # 0.005
```

+0,103

de 5 a 400 árboles

+0,005

de 100 a 400 árboles

CONSULTA · ACLARACIONES · EJERCICIO 2

Leer los rendimientos decrecientes (ejercicio 2)

oob_score=True

Guarda en **oob_decision_function_** la **probabilidad OOB** de cada reserva de train: sin separar datos.

np.nan_to_num(..., nan=0.5)

Con pocos árboles, algunas reservas no quedan fuera en ninguno (NaN): se rellenan con 0,5, «**no sé**».

oob_auc[-1] - oob_auc[0]

El último AUC (400 árboles) menos el primero (5): **+0,103**. **n_arboles.index(100)** da la posición del de 100.

Unas 20 veces menos

De 100 a 400, **+0,005**: cuatro veces más árboles cuestan cuatro veces más tiempo. Con 100–200 suele sobrar.

CONSULTA · EJERCICIO 3 (CASA)

La variable principal y las irrelevantes (ejercicio 3)

```
modelo_bosque = Pipeline([("prep", prep), ("rf", RandomForestClassifier(n_estimators=300, min_samples_leaf=5, random_state=SEMILLA,
    n_jobs=2))]).fit(X_train, y_train)
perm = permutation_importance(modelo_bosque, X_test, y_test, scoring="roc_auc", n_repeats=5, random_state=SEMILLA)
imp_perm = pd.Series(perm.importances_mean, index=X_test.columns).sort_values(ascending=False)
variable_top = imp_perm.index[0]                  # 'antelacion_dias'
n_irrelevantes = int((imp_perm < 0.002).sum())    # 6
```

0,110

caída del AUC al barajar **antelacion_dias**

6

columnas casi irrelevantes (de 15)

CONSULTA · ACLARACIONES · EJERCICIO 3

Leer la permutación (ejercicio 3, casa)

permutation_importance

Baraja cada columna del **test** (5 veces) y mide cuánto **cae el AUC**; **importances_mean** es la media.

imp_perm.index[0]

La Serie está ordenada de mayor a menor: el **primer nombre** es la variable principal.

Arriba del todo

antelacion_dias (0,110), **deposito** (0,047), **canal** (0,043) y **tarifa** (0,031).

6 irrelevantes

cancelaciones_previas, **noches**, **adultos**, **ninos**, **regimen** y **pais_origen**: una caída **negativa** (≈ 0) = **ruido**, no aporta.

CONSULTA · EJEMPLO (CASA)

Tiempo de ajuste: árbol frente a bosque

```
t0 = time.perf_counter()
RandomForestClassifier(n_estimators=300, min_samples_leaf=5, random_state=SEMILLA, n_jobs=2).fit(Xtr, y_train)
t_bosque = time.perf_counter() - t0    # ≈ 0.8 s
t0 = time.perf_counter()
DecisionTreeClassifier(min_samples_leaf=80, random_state=SEMILLA).fit(Xtr, y_train)
t_arbol = time.perf_counter() - t0     # ≈ 0.012 s
```

12 ms

árbol

821 ms

bosque (≈ 67 veces más)

CONSULTA · ACLARACIONES · EJEMPLO DEL TIEMPO

Leer el tiempo de ajuste (casa)

time.perf_counter()

Marca el instante: se llama **antes y después** de **.fit(...)** y se **resta**.

Por qué tanto más

Son **300 árboles**, y cada uno es más profundo que el podado (**min_samples_leaf=5** frente a 80).

Cifras que cambian

Los ms y el factor **varían** con el ordenador y la ejecución; lo habitual es **50–70 veces más**.

¿Compensa?

Por **0,018** de AUC, 800 ms son poca cosa; con **millones de datos**, el coste se dispararía.

CONSULTA · MINI-RETO (CASA)

Duelo: árbol, bosque y logística (mini-reto)

```
filas = {n: cross_val_score(m, X_train, y_train, cv=cv, scoring="roc_auc") for n, m in modelos.items()}
duelo = pd.DataFrame({n: {"auc_media": s.mean(), "desviacion": s.std()} for n, s in filas.items()}).T.round(3)
mejor_modelo = duelo["auc_media"].idxmax()    # 'logistica'
```

Modelo (validación cruzada, 5 folds)	AUC medio	Desviación
arbol	0,716	0,016
bosque	0,741	0,014
logistica	**0,758**	0,012

Más complejo no siempre es mejor: aquí los datos siguen una relación casi logística.

CONSULTA · ACLARACIONES · MINI-RETO

Leer el duelo (mini-reto, casa)

cross_val_score(..., cv=cv)

Devuelve **5 AUC**, uno por fold; **.mean()** y **.std()** dan la media y la **dispersión**.

idxmax()

Devuelve el **nombre** de la fila con mayor AUC medio: **'logistica'**.

La ventaja es pequeña

Logística − bosque = 0,017, del **mismo orden** que la desviación entre folds (≈ 0,012–0,014).

Cuál elegir

La logística: rinde igual o mejor, es más barata y **se explica** (odds ratio). El bosque pierde en las tres cosas.